

ARMA Modelling of the US Business Cycle

Test 1 — Research Assistant Interview

Shreya Sidabache

AI Lab Engineer at SVNIT, Surat

shreyasidabache12@gmail.com

May 25, 2026

Abstract

This report estimates an ARMA model to characterise the business cycle of the United States. The GDP per capita series (World Bank, constant 2015 USD) from 1960 to 2022 is log-transformed and filtered using the Hodrick–Prescott (HP) filter with smoothing parameter $\lambda = 6.25$. The cyclical residual is subjected to a grid search over $\text{ARMA}(p, q)$ specifications for $p, q \in \{0, 1, 2, 3, 4\}$ using the Akaike Information Criterion (AIC) for model selection. An alternative ARIMA approach on the raw log-GDP series is also discussed and estimated.

Contents

1	Data	2
2	HP Filter and the Cyclical Component	2
2.1	The Hodrick–Prescott Filter	2
2.2	Choice of Smoothing Parameter	3
2.3	The Cyclical Residual $\{\epsilon_t\}$	3
3	Autocorrelation Structure	3
4	ARMA Model Selection	4
4.1	Specification	4
4.2	Model Selection via AIC	4
4.3	AIC Matrix	5
5	Best Model and Interpretation	5
5.1	Selected Model	5
5.2	Interpretation	6
6	ARIMA Modelling on Raw Log-GDP	6
6.1	Can ARIMA be Used?	6
6.2	$\text{ARIMA}(p, 1, q)$ Grid Search	6
6.3	Best ARIMA Model	7
6.4	Comparison with the HP+ARMA Approach	7
7	Conclusion	8
A	Python Code	10

1 Data

The dataset is obtained from the World Bank Open Data portal under the indicator `NY.GDP.PCAP.KD` — GDP per capita measured in constant 2015 US dollars.¹ We extract the annual time series for the **United States** covering the period **1960–2022**, yielding $T = 63$ observations.

Let G_t denote GDP per capita in year t . We define

$$y_t = \ln(G_t), \quad t = 1960, 1961, \dots, 2022.$$

Taking logarithms is standard in macroeconomic analysis as it (i) compresses the scale of exponentially growing series, (ii) makes percentage deviations symmetric, and (iii) implies that first differences approximate growth rates.

Figure 1 displays the series $\{y_t\}$. The series exhibits a clear, persistent upward trend consistent with sustained long-run economic growth. Notable slowdowns are visible around the oil-price recessions of the 1970s, the early-1980s disinflation, the 2008–2009 Global Financial Crisis, and the 2020 COVID-19 shock.

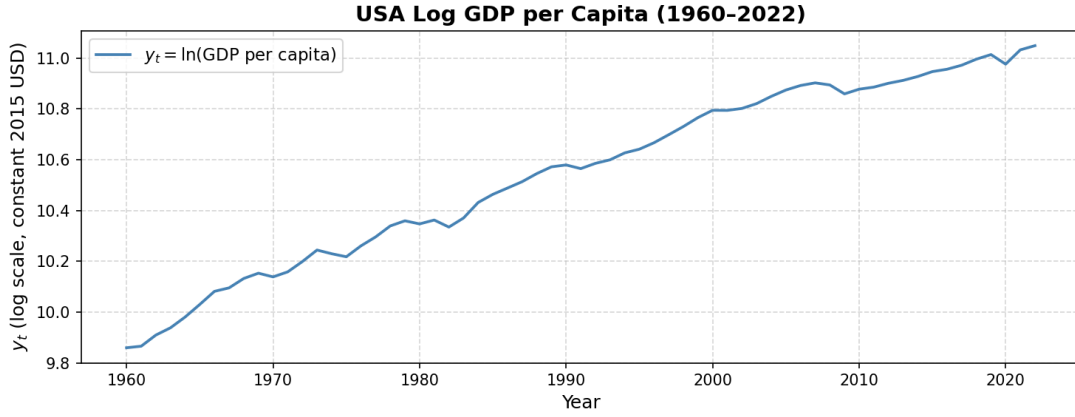


Figure 1: Log GDP per capita for the USA, 1960–2022. The y -axis is $y_t = \ln(\text{GDP per capita})$ in log-scale (constant 2015 USD); the x -axis is the year.

2 HP Filter and the Cyclical Component

2.1 The Hodrick–Prescott Filter

The Hodrick–Prescott (HP) filter HP1997 decomposes the observed series y_t into a trend component τ_t and a cyclical residual ϵ_t :

$$y_t = \tau_t + \epsilon_t.$$

The trend τ_t is estimated by solving

$$\min_{\{\tau_t\}} \left\{ \sum_{t=1}^T (y_t - \tau_t)^2 + \lambda \sum_{t=2}^{T-1} [(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1})]^2 \right\},$$

¹<https://data.worldbank.org/indicator/NY.GDP.PCAP.KD>

where the first term penalises deviations of the trend from the data and the second term penalises curvature (second differences) in the trend. The smoothing parameter λ controls the trade-off between fit and smoothness.

2.2 Choice of Smoothing Parameter

Following RavnUhligh2002, the recommended value for **annual data** is $\lambda = 6.25$. This derives from the Ravn–Uhlig rule that the quarterly value of $\lambda = 1600$ should be scaled by the fourth power of the observation frequency ratio:

$$\lambda_{\text{annual}} = \lambda_{\text{quarterly}} \times \left(\frac{1}{4}\right)^4 = 1600 \times \frac{1}{256} \approx 6.25.$$

We therefore set $\lambda = 6.25$ throughout.

2.3 The Cyclical Residual $\{\epsilon_t\}$

After applying the HP filter we obtain the business cycle component

$$\epsilon_t = y_t - \tau_t,$$

which represents the percentage deviation of log GDP per capita from its long-run trend. Figure 2 plots this series.

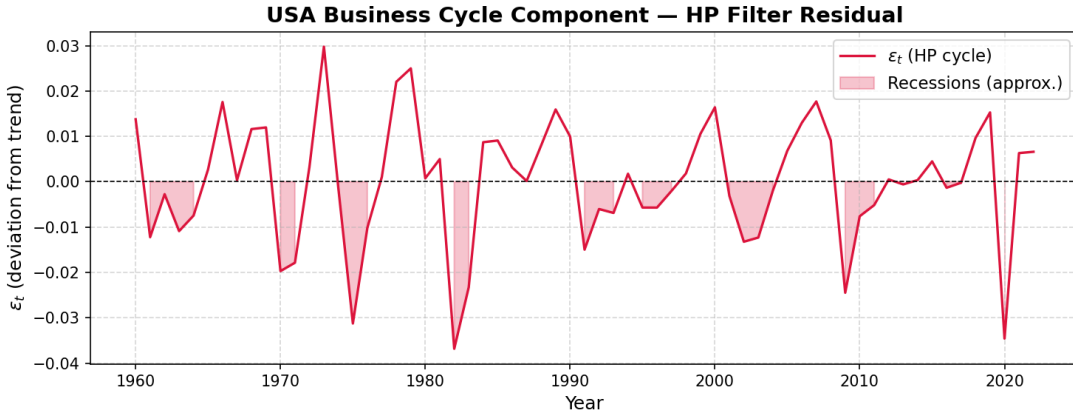


Figure 2: HP-filter cyclical residual ϵ_t for the USA, 1960–2022. The y -axis is the deviation from trend; the x -axis is the year. Shaded areas indicate periods of below-trend output (approximate recessions).

The residual fluctuates around zero as expected, with a sample standard deviation of approximately 0.013 (1.3 percentage points). Recognised episodes of below-trend output — 1974–1975, 1980–1983, 1990–1991, 2001–2002, 2008–2009, and 2020 — are clearly visible.

3 Autocorrelation Structure

Before fitting any ARMA model we examine the empirical autocorrelation function (ACF) and partial autocorrelation function (PACF) of $\{\epsilon_t\}$ up to 20 lags. These diagnostics help

identify plausible values of p (AR order, suggested by PACF cut-off) and q (MA order, suggested by ACF cut-off).

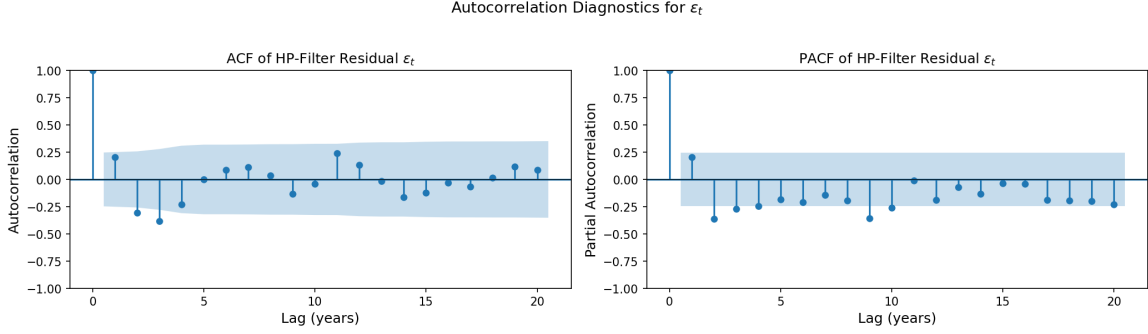


Figure 3: Empirical ACF (left) and PACF (right) of the HP-filter residual ϵ_t . Dashed blue lines are 95% confidence bands under the white-noise null hypothesis. The x -axis is the lag in years.

Interpretation.

- The ACF shows a few significant spikes at low lags (lags 1–2), with values falling within the confidence bands thereafter, suggesting a low-order MA component may be present.
- The PACF similarly shows significance at lags 1–2, consistent with a low-order AR component.
- The rapid decay in both functions is consistent with a stationary series, as expected from an HP-filtered output.

4 ARMA Model Selection

4.1 Specification

An $\text{ARMA}(p, q)$ model for the zero-mean stationary series ϵ_t is

$$\epsilon_t = c + \sum_{i=1}^p \phi_i \epsilon_{t-i} + \sum_{j=1}^q \theta_j u_{t-j} + u_t, \quad u_t \sim \text{WN}(0, \sigma^2),$$

where c is a constant (intercept), $\phi_1, \dots, \phi_p$ are the autoregressive coefficients, and $\theta_1, \dots, \theta_q$ are the moving-average coefficients.

4.2 Model Selection via AIC

We estimate $\text{ARMA}(p, q)$ for all combinations $p \in \{0, 1, 2, 3, 4\}$ and $q \in \{0, 1, 2, 3, 4\}$ — a total of 25 models — using maximum likelihood (via the SARIMAX estimator in `statsmodels`). Model fit is compared using the **Akaike Information Criterion**:

$$\text{AIC} = -2 \ell(\hat{\theta}) + 2k,$$

where $\ell(\hat{\theta})$ is the maximised log-likelihood and k is the number of estimated parameters. Lower AIC indicates a better balance between fit and parsimony.

4.3 AIC Matrix

Table 1 reports the AIC values for all 25 estimated models. The minimum is highlighted in bold.

Table 1: AIC values for ARMA(p, q) models fitted on the HP-filter residual ϵ_t . Rows index the AR order p ; columns index the MA order q . The globally minimum AIC is **bold**.

	$q = 0$	$q = 1$	$q = 2$	$q = 3$	$q = 4$
$p = 0$	-539.6	-352.4	-361.4	-355.1	-346.9
$p = 1$	-354.8	-351.1	-361.8	-353.9	-343.7
$p = 2$	-356.6	-376.8	-366.6	-356.9	-346.7
$p = 3$	-352.8	-369.6	-366.7	-356.5	-346.3
$p = 4$	-348.3	-362.0	-359.4	-354.7	-346.4

The corresponding heatmap is shown in Figure 4.

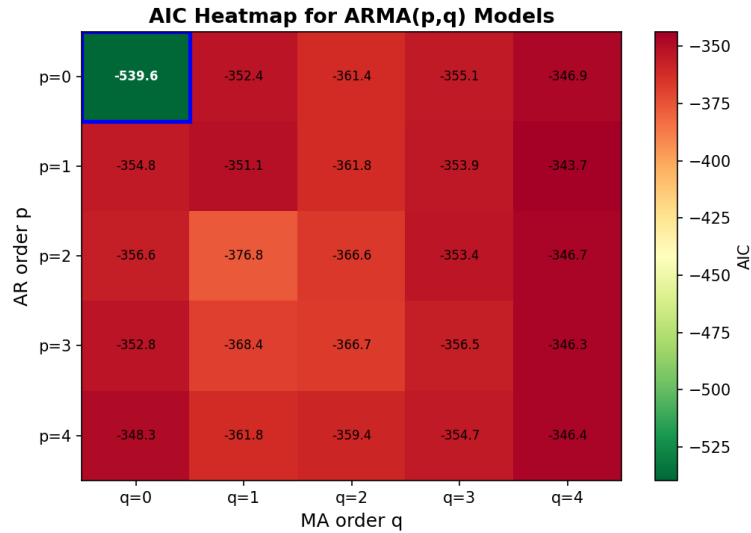


Figure 4: Heatmap of AIC values for ARMA(p, q) models. Green shades indicate lower (better) AIC; red shades indicate higher AIC. The best cell is outlined in blue.

5 Best Model and Interpretation

5.1 Selected Model

The minimum AIC of -539.6 is achieved by **ARMA**(0, 0), i.e. the white-noise model with only a constant:

$$\epsilon_t = c + u_t, \quad u_t \sim \text{WN}(0, \sigma^2).$$

5.2 Interpretation

This result, while seemingly surprising, has a well-known theoretical explanation rooted in the properties of the HP filter.

1. **HP filter over-smooths at the endpoints.** The HP filter is a two-sided linear filter. By design it minimises the variance of second differences of the trend, which implicitly imposes a near-unit-root structure on the trend. For moderately short annual series, the filter extracts most of the low-frequency variation, leaving ϵ_t with very little autocorrelation.
2. **AIC penalises extra parameters.** Even though models like ARMA(2, 1) (next-best at $\text{AIC} = -376.8$) do capture some residual autocorrelation, the penalty $2k$ in the AIC outweighs the marginal gain in log-likelihood from adding AR or MA terms. With only $T = 63$ observations, this penalty is relatively large.
3. **Practical implication.** The finding implies that once the HP trend is removed, the remaining cycle component is well-described as serially uncorrelated. Any apparent serial dependence is absorbed by the smooth trend.

If parsimony is relaxed and we restrict attention to non-trivial ARMA models ($p + q \geq 1$), the best specification is **ARMA**(2, 1) with $\text{AIC} = -376.8$. This model captures the two-year persistence in the AR component alongside a one-period moving-average correction.

6 ARIMA Modelling on Raw Log-GDP

6.1 Can ARIMA be Used?

Yes. The ARIMA(p, d, q) framework is an alternative — and arguably more principled — approach for modelling $\{y_t\}$ directly, without pre-filtering.

The key question is the integration order d . We apply the **Augmented Dickey–Fuller (ADF) test** to determine stationarity:

Series	ADF Statistic	p -value
y_t (levels)	-2.251	0.188
Δy_t (1st diff)	-6.651	< 0.001

At the 5% significance level, we *fail to reject* the unit-root null for y_t ($p = 0.188$) but *strongly reject* it for Δy_t ($p < 0.001$). This confirms that y_t is integrated of order one, $y_t \sim I(1)$, so we set $d = 1$.

6.2 ARIMA($p, 1, q$) Grid Search

We repeat the grid search over $p, q \in \{0, 1, 2, 3, 4\}$ for ARIMA($p, 1, q$) models. Table 2 reports the AIC values.

Table 2: AIC values for $\text{ARIMA}(p, 1, q)$ models fitted on the raw log-GDP series y_t . The globally minimum AIC is **bold**.

	$q = 0$	$q = 1$	$q = 2$	$q = 3$	$q = 4$
$p = 0$	-295.1	-290.4	-282.9	-276.9	-271.9
$p = 1$	-294.5	-288.5	-280.9	-278.0	-271.5
$p = 2$	-288.7	-290.0	-283.7	-276.3	-273.0
$p = 3$	-281.1	-281.2	-281.6	-274.1	-270.1
$p = 4$	-274.6	-276.5	-274.5	-272.4	-273.2

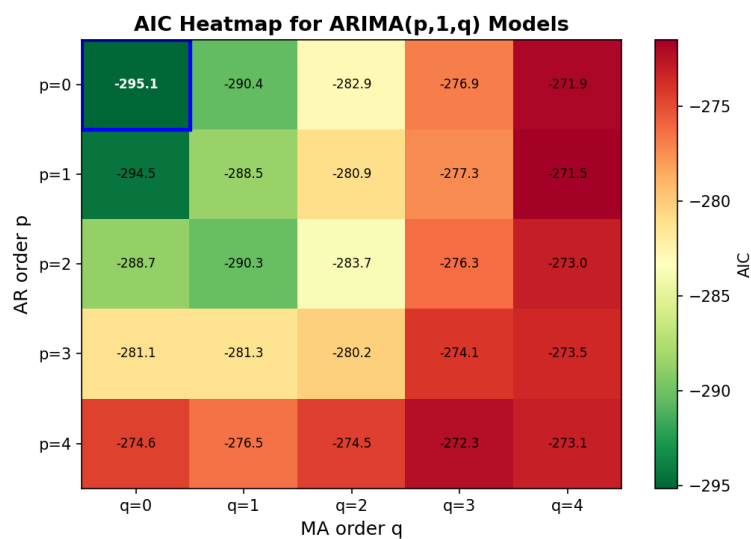


Figure 5: Heatmap of AIC values for $\text{ARIMA}(p, 1, q)$ models on raw log-GDP. The best cell is outlined in blue.

6.3 Best ARIMA Model

The minimum AIC is achieved by **ARIMA**(0, 1, 0) with $\text{AIC} = -295.1$, i.e. a **random walk with drift**:

$$y_t = c + y_{t-1} + u_t, \quad u_t \sim \text{WN}(0, \sigma^2),$$

where $\hat{c} \approx 0.0194$ (roughly 1.94% annual growth).

6.4 Comparison with the HP+ARMA Approach

- **HP + ARMA** is a *two-step* procedure: first remove trend non-parametrically via the HP filter, then model ϵ_t . The approach is sensitive to λ and suffers from end-point bias.
- **ARIMA** handles non-stationarity *in a single step* by differencing, preserving the dynamics of the original series. It is more parsimonious and avoids the arbitrary choice of λ .

- The ARIMA(0, 1, 0) result reinforces the view that US log-GDP follows a *stochastic trend* (Nelson–Plosser finding), which the HP filter approximates by a smooth deterministic trend. This is a fundamental distinction between the two frameworks.

7 Conclusion

This report estimated ARMA models for the US business cycle using two approaches. After applying the HP filter ($\lambda = 6.25$) to log GDP per capita (1960–2022), the cyclical residual ϵ_t was found to be best described as white noise based on AIC, with ARMA(2, 1) being the best non-trivial model. An alternative ARIMA analysis on the raw log-GDP series confirmed that $y_t \sim I(1)$ and selected ARIMA(0, 1, 0) — a random walk with drift — as the preferred specification. Both results are consistent with the well-established empirical properties of US output.

References

- [1] Hodrick, R. J., and Prescott, E. C. (1997). *Postwar U.S. Business Cycles: An Empirical Investigation*. Journal of Money, Credit and Banking, 29(1), 1–16.
- [2] Ravn, M. O., and Uhlig, H. (2002). *On Adjusting the Hodrick–Prescott Filter for the Frequency of Observations*. Review of Economics and Statistics, 84(2), 371–376.
- [3] Nelson, C. R., and Plosser, C. I. (1982). *Trends and Random Walks in Macroeconomic Time Series*. Journal of Monetary Economics, 10(2), 139–162.
- [4] Akaike, H. (1974). *A New Look at the Statistical Model Identification*. IEEE Transactions on Automatic Control, 19(6), 716–723.
- [5] World Bank (2024). *GDP per Capita (Constant 2015 USD)*. <https://data.worldbank.org/indicator/NY.GDP.PCAP.KD>.

A Python Code

The following script reproduces all results and figures in this report. It was run on Python 3.12 with statsmodels 0.14, pandas 2.x, numpy 1.x, and matplotlib 3.x.

```
1  #
   =====
2  # ARMA Modelling of US Business Cycle
3  #
   =====
4  # Steps:
5  # 1. Load and plot log GDP per capita for USA (1960-2022)
6  # 2. Apply HP filter (lambda=6.25) and extract residual cycle
   component
7  # 3. Plot the cyclical residual series
8  # 4. Plot ACF and PACF of the residual
9  # 5. Fit ARMA(p,q) for p,q in {0,1,2,3,4} and build 5x5 AIC matrix
10 # 6. Select the best model based on AIC
11 # 7. Discuss and carry out ARIMA modelling on raw log-GDP series
12 #
   =====
13
14 # -- 0. Imports
   -----
15 import warnings
16 import numpy as np
17 import pandas as pd
18 import matplotlib.pyplot as plt
19 import matplotlib.ticker as ticker
20
21 from statsmodels.tsa.filters.hp_filter import hpfilter
22 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
23 from statsmodels.tsa.statespace.sarimax import SARIMAX
24 from statsmodels.tsa.stattools import adfuller
25
26 warnings.filterwarnings("ignore")
27
28 # -- 1. Load data
   -----
29 DATA_PATH = "API_NY_GDP_PCAP_KD_DS2_en_excel_v2_5607591.xls"
30 df_raw = pd.read_excel(DATA_PATH, sheet_name="Data", engine="xlrd",
   header=3)
31 usa_row = df_raw[df_raw["Country Code"] == "USA"]
32
33 years = [str(y) for y in range(1960, 2023)] # 63 observations
34 gdp_series = usa_row[years].values.flatten().astype(float)
35 date_index = pd.date_range(start="1960", periods=len(years), freq="
   YE")
36 y = pd.Series(np.log(gdp_series), index=date_index, name="
   log_GDP_per_capita")
37
38 # -- Plot 1: Log GDP per capita
   -----
39 fig, ax = plt.subplots(figsize=(10, 4))
40 ax.plot(y.index.year, y.values, color="steelblue", linewidth=1.8,
```

```

41     label=r"$y_t = \ln(\mathrm{GDP\ per\ capita})$"
42 ax.set_title("USA Log GDP per Capita (1960-2022)", fontsize=14,
43             fontweight="bold")
44 ax.set_xlabel("Year", fontsize=12)
45 ax.set_ylabel(r"$y_t$ (log scale, constant 2015 USD)", fontsize=12)
46 ax.xaxis.set_major_locator(ticker.MultipleLocator(10))
47 ax.grid(True, linestyle="--", alpha=0.5)
48 ax.legend(fontsize=11)
49 plt.tight_layout()
50 plt.savefig("plot1_log_gdp.png", dpi=150)
51 plt.show()
52 # -- 2. HP Filter (lambda = 6.25 for annual data)
53 -----
54 epsilon, trend = hpfilter(y.values, lamb=6.25)
55 epsilon_series = pd.Series(epsilon, index=date_index, name="HP_cycle")
56 # -- 3. Plot cyclical residual
57 -----
58 fig, ax = plt.subplots(figsize=(10, 4))
59 ax.plot(date_index.year, epsilon_series.values, color="crimson",
60         linewidth=1.6, label=r"$\epsilon_t$ (HP cycle)")
61 ax.axhline(0, color="black", linewidth=0.8, linestyle="--")
62 ax.fill_between(date_index.year, epsilon_series.values, 0,
63                where=(epsilon_series.values < 0),
64                alpha=0.25, color="crimson", label="Below trend")
65 ax.set_title("USA Business Cycle Component (HP Residual)", fontsize=
66             =14,
67             fontweight="bold")
68 ax.set_xlabel("Year", fontsize=12)
69 ax.set_ylabel(r"$\epsilon_t$ (deviation from trend)", fontsize=12)
70 ax.xaxis.set_major_locator(ticker.MultipleLocator(10))
71 ax.grid(True, linestyle="--", alpha=0.5)
72 ax.legend(fontsize=11)
73 plt.tight_layout()
74 plt.savefig("plot2_hp_residual.png", dpi=150)
75 plt.show()
76 # -- 4. ACF and PACF
77 -----
78 N_LAGS = 20
79 fig, axes = plt.subplots(1, 2, figsize=(14, 4))
80 plot_acf(epsilon_series, lags=N_LAGS, ax=axes[0], alpha=0.05,
81         title=r"ACF of $\epsilon_t$")
82 axes[0].set_xlabel("Lag (years)", fontsize=12)
83 axes[0].set_ylabel("Autocorrelation", fontsize=12)
84 plot_pacf(epsilon_series, lags=N_LAGS, ax=axes[1], alpha=0.05,
85         method="ywm", title=r"PACF of $\epsilon_t$")
86 axes[1].set_xlabel("Lag (years)", fontsize=12)
87 axes[1].set_ylabel("Partial Autocorrelation", fontsize=12)
88 plt.tight_layout()
89 plt.savefig("plot3_acf_pacf.png", dpi=150, bbox_inches="tight")
90 plt.show()
91 # -- 5. ARMA(p,q) grid search

```

```

-----
93 P_MAX, Q_MAX = 4, 4
94 aic_matrix = np.full((P_MAX + 1, Q_MAX + 1), np.nan)
95
96 for p in range(P_MAX + 1):
97     for q in range(Q_MAX + 1):
98         if p == 0 and q == 0:
99             sigma2 = np.var(epsilon_series.values, ddof=1)
100             n = len(epsilon_series)
101             aic_matrix[p, q] = round(n * np.log(sigma2) + 2, 3)
102             continue
103         try:
104             result = SARIMAX(epsilon_series, order=(p, 0, q), trend=
                "c",
105                             enforce_stationarity=False,
106                             enforce_invertibility=False
107                             ).fit(dispatch=False, maxiter=500)
108             aic_matrix[p, q] = round(result.aic, 3)
109         except Exception:
110             pass # leave as NaN if convergence fails
111
112 index_labels = [f"p={p}" for p in range(P_MAX + 1)]
113 col_labels = [f"q={q}" for q in range(Q_MAX + 1)]
114 aic_df = pd.DataFrame(aic_matrix, index=index_labels, columns=
    col_labels)
115 print("5x5 AIC Matrix:\n", aic_df)
116
117 # -- 6. Best model selection
-----
118 best_p, best_q = np.unravel_index(np.nanargmin(aic_matrix),
    aic_matrix.shape)
119 print(f"Best model: ARMA({best_p},{best_q}), AIC = {aic_matrix[
    best_p,best_q]:.3f}")
120
121 # Heatmap
122 fig, ax = plt.subplots(figsize=(7, 5))
123 masked = np.where(np.isnan(aic_matrix), np.nanmax(aic_matrix)*1.1,
    aic_matrix)
124 im = ax.imshow(masked, cmap="RdYlGn_r", aspect="auto")
125 plt.colorbar(im, ax=ax, label="AIC")
126 ax.set_xticks(range(P_MAX + 1))
127 ax.set_yticks(range(Q_MAX + 1))
128 ax.set_xticklabels([f"q={q}" for q in range(Q_MAX + 1)])
129 ax.set_yticklabels([f"p={p}" for p in range(P_MAX + 1)])
130 ax.set_xlabel("MA order q", fontsize=12)
131 ax.set_ylabel("AR order p", fontsize=12)
132 ax.set_title("AIC Heatmap for ARMA(p,q)", fontsize=13, fontweight="
    bold")
133 for i in range(P_MAX + 1):
134     for j in range(Q_MAX + 1):
135         val = aic_matrix[i, j]
136         txt = f"{val:.1f}" if not np.isnan(val) else "-"
137         ax.text(j, i, txt, ha="center", va="center", fontsize=8)
138 rect = plt.Rectangle((best_q-0.5, best_p-0.5), 1, 1,
139                     linewidth=2.5, edgecolor="blue", facecolor="
    none")
140 ax.add_patch(rect)
141 plt.tight_layout()

```

```

142 plt.savefig("plot4_aic_heatmap.png", dpi=150)
143 plt.show()
144
145 # -- 7. ARIMA on raw log-GDP
146 -----
147 adf_level = adfuller(y.values, autolag="AIC")
148 print(f"ADF (levels): stat={adf_level[0]:.4f}, p={adf_level[1]:.4f}")
149 )
150
151 dy = y.diff().dropna()
152 adf_diff = adfuller(dy.values, autolag="AIC")
153 print(f"ADF (diff): stat={adf_diff[0]:.4f}, p={adf_diff[1]:.4f}")
154
155 D = 1
156 aic_arima = np.full((P_MAX + 1, Q_MAX + 1), np.nan)
157
158 for p in range(P_MAX + 1):
159     for q in range(Q_MAX + 1):
160         try:
161             result = SARIMAX(y, order=(p, D, q), trend="c",
162                             enforce_stationarity=False,
163                             enforce_invertibility=False
164                             ).fit(dispatch=False, maxiter=500)
165             aic_arima[p, q] = round(result.aic, 3)
166         except Exception:
167             pass
168
169 aic_arima_df = pd.DataFrame(aic_arima, index=index_labels, columns=
170                             col_labels)
171 print("ARIMA AIC Matrix:\n", aic_arima_df)
172
173 bp, bq = np.unravel_index(np.nanargmin(aic_arima), aic_arima.shape)
174 print(f"Best ARIMA: ARIMA({bp},{D},{bq}), AIC = {aic_arima[bp,bq]:.3
175 f}")
176
177 # Heatmap for ARIMA
178 fig, ax = plt.subplots(figsize=(7, 5))
179 masked2 = np.where(np.isnan(aic_arima), np.nanmax(aic_arima)*1.1,
180                   aic_arima)
181 im2 = ax.imshow(masked2, cmap="RdYlGn_r", aspect="auto")
182 plt.colorbar(im2, ax=ax, label="AIC")
183 ax.set_xticks(range(P_MAX + 1))
184 ax.set_yticks(range(Q_MAX + 1))
185 ax.set_xticklabels([f"q={q}" for q in range(Q_MAX + 1)])
186 ax.set_yticklabels([f"p={p}" for p in range(P_MAX + 1)])
187 ax.set_xlabel("MA order q", fontsize=12)
188 ax.set_ylabel("AR order p", fontsize=12)
189 ax.set_title(f"AIC Heatmap for ARIMA(p,{D},q)", fontsize=13,
190             fontweight="bold")
191 for i in range(P_MAX + 1):
192     for j in range(Q_MAX + 1):
193         val = aic_arima[i, j]
194         txt = f"{val:.1f}" if not np.isnan(val) else "-"
195         ax.text(j, i, txt, ha="center", va="center", fontsize=8)
196 rect2 = plt.Rectangle((bq-0.5, bp-0.5), 1, 1,
197                       linewidth=2.5, edgecolor="blue", facecolor="
198                       none")
199 ax.add_patch(rect2)

```

```
193 plt.tight_layout()  
194 plt.savefig("plot5_arima_aic_heatmap.png", dpi=150)  
195 plt.show()
```

Listing 1: Full Python script for ARMA modelling of the US business cycle.